

A faint, light gray world map is visible in the background of the slide, centered behind the title text.

计算机组织与结构复习课

课程章节内容

- **1 计算机系统概论**
- **2 运算方法和运算器**
- **3 存储系统**
- **4 指令系统**
- **5 中央处理器**
- **6 输入输出系统**

第一章 计算机系统概述

第一章 计算机系统概述

◦ 1、计算机的性能指标

总线宽度：一般指CPU中运算器与存储器之间进行互连的内部总线二进制位数。

存储器容量：存储器中所有存储单元的总数目，通常用KB、

CPI：表示每条指令周期数，即执行一条指令所需的平均时钟周期数。

MIPS：表示每秒百万条指令数。

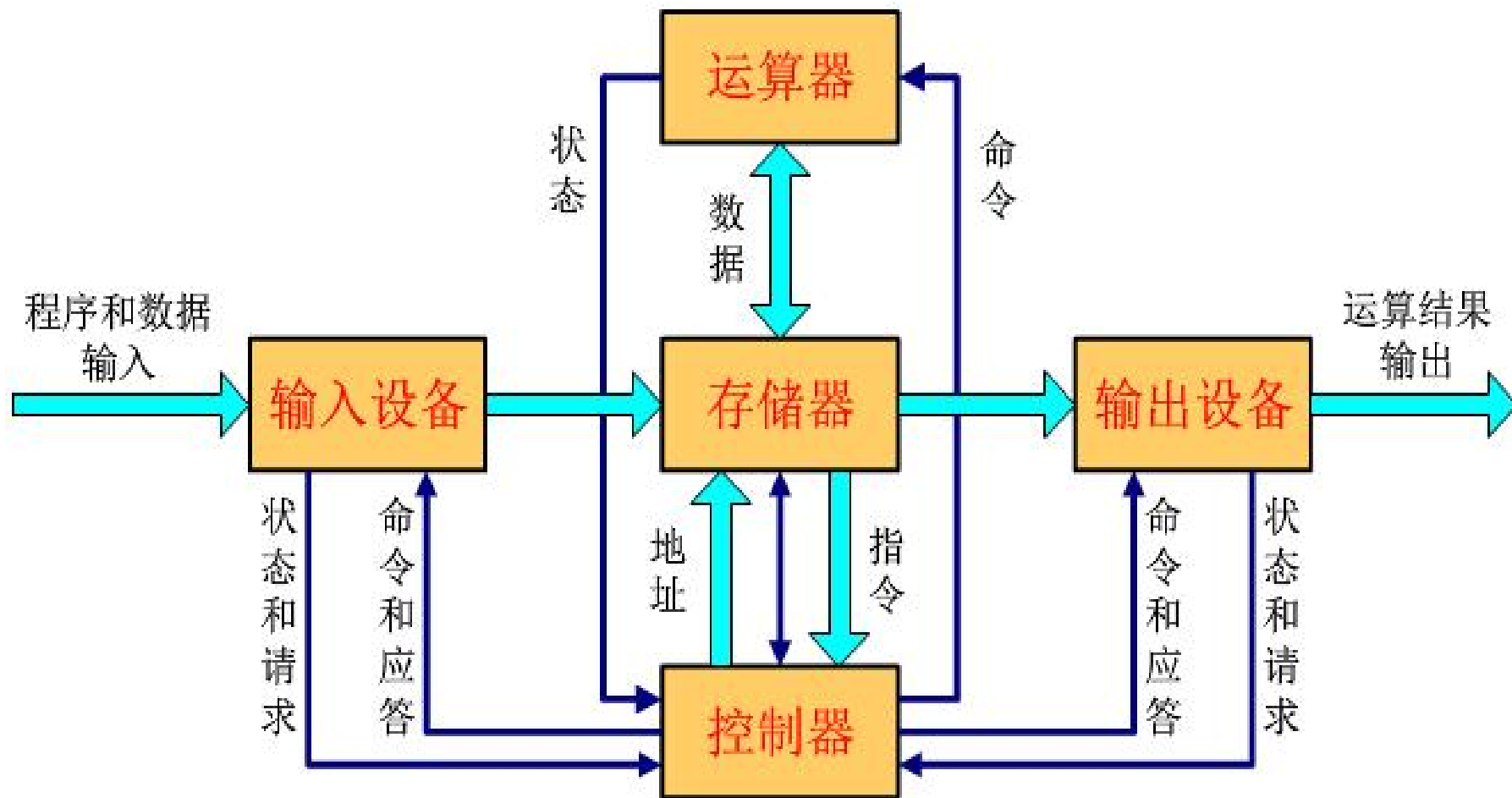
因为每条指令执行时间不同，所以MIPS总是一个平均值。

第一章 计算机系统概述

◦ 2、冯·诺依曼结构的主要思想

- ① 计算机应由运算器、控制器、存储器、输入设备和输出设备五个基本部件组成。
- ② 各基本部件的功能是：
 - **存储器**不仅能存放数据，而且也能存放指令，形式上两者没有区别，但计算机应能区分数据还是指令；
 - **控制器**应能自动执行指令；
 - **运算器**应能进行加/减/乘/除四种基本算术运算，并且也能进行一些逻辑运算和附加运算；
 - 操作人员可以通过**输入设备**、**输出设备**和主机进行通信。
- ③ 内部以**二进制表示**指令和数据。每条指令由操作码和地址码两部分组成。操作码指出操作类型，地址码指出操作数的地址。由一串指令组成程序。
- ④ 采用“**存储程序**”工作方式。

冯.诺依曼结构计算机模型



◦ 3、计算机的基本组成与基本功能

◦ 什么是计算机？

- 计算机是一种能对数字化信息进行自动、高速算术和逻辑运算的处理装置。

◦ 计算机的基本部件及功能：

- 运算器（数据运算）：ALU、GPRs、标志寄存器等
- 存储器（数据存储）：存储阵列、地址译码器、读写控制电路
- 总线（数据传送）：数据(MDR)、地址(MAR)和控制线
- 控制器（控制）：对指令译码生成控制信号

◦ 计算机实现的所有任务都是通过执行一条一条指令完成的！

4、指令和数据

- **程序启动前**，指令和数据都存放在存储器中，形式上没有差别，都是0/1序列
- 采用“**存储程序**”工作方式：
 - 程序由指令组成，程序被启动后，计算机能自动取出一条一条指令执行，在执行过程中无需人的干预。
- **指令执行过程中**，指令和数据被从存储器取到CPU，存放在CPU内的寄存器中，指令在**IR**中，数据在**GPR**中。

指令中需给出的信息：

操作性质（操作码）

源操作数1 或/和 源操作数2 （立即数、寄存器编号、存储地址）

目的操作数地址 （寄存器编号、存储地址）

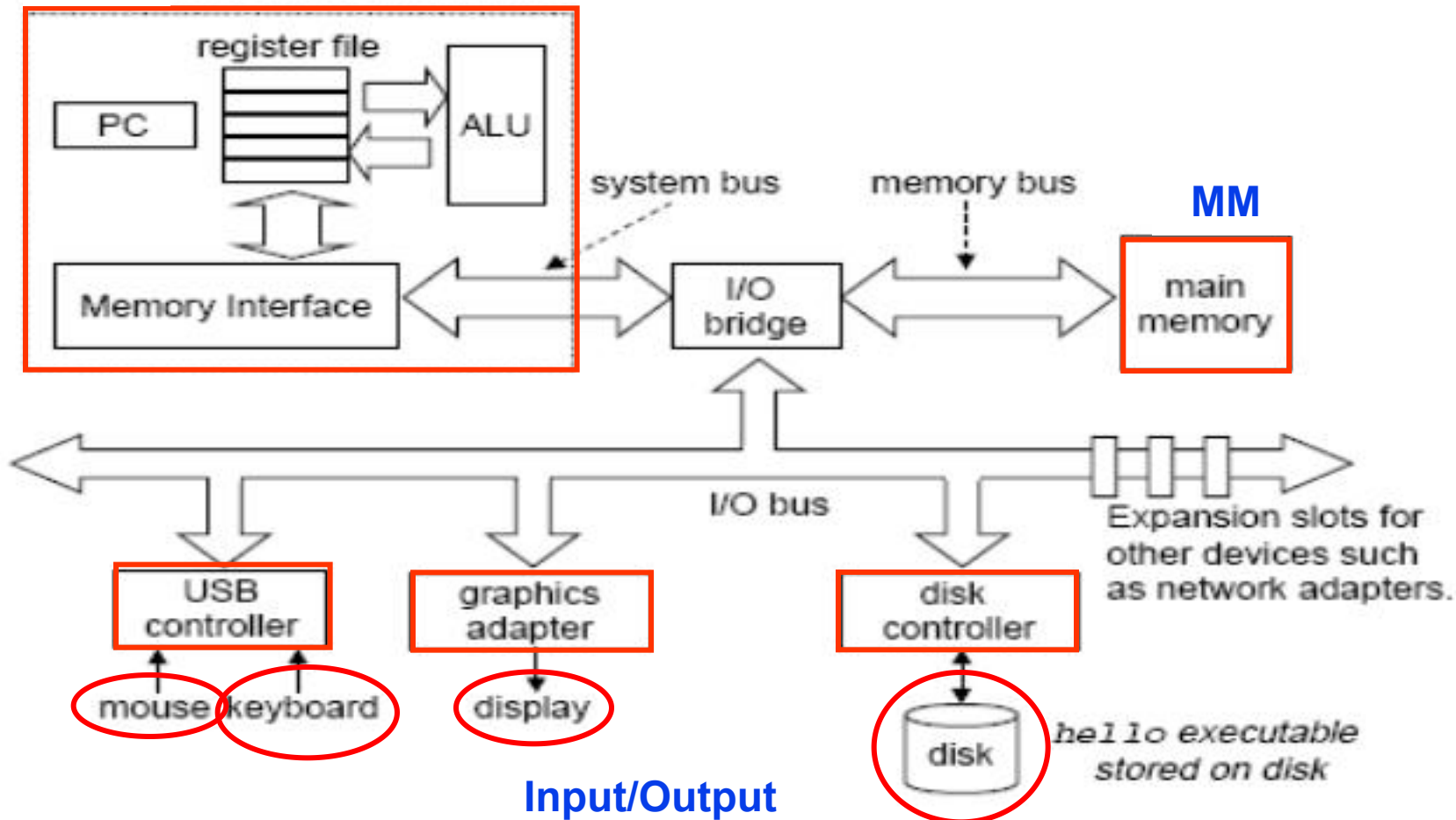
存储地址的描述与操作数的数据结构有关！

5、软件 Software

- System software(系统软件) - 简化编程，并使硬件资源被有效利用
 - 操作系统 (Operating System) : 硬件资源管理，用户接口
 - 语言处理系统：翻译程序+ Linker, Debug, etc ...
 - 翻译程序(Translator)有三类：
 - 汇编程序(Assembler): 汇编语言源程序→机器目标程序
 - 编译程序(Compiler): 高级语言源程序→汇编/机器目标程序
 - 解释程序(Interpreter): 将高级语言语句逐条翻译成机器指令并立即执行, 不生成目标文件。
 - 其他实用程序: 如：磁盘碎片整理程序、备份程序等
- Application software(应用软件) - 解决具体应用问题/完成具体应用
 - 各类媒体处理程序：Word/ Image/ Graphics/...
 - 管理信息系统 (MIS)
 - Game, ...

6、一个典型系统的硬件组成

CPU



PC: 程序计数器; ALU: 算术/逻辑单元; USB: 通用串行总线

第二章 运算方法和运算器

1、信息的二进制编码

◆ 计算机的外部信息与内部机器级数据

◆ 机器级数据分两大类：

- 数值数据：无符号整数、带符号整数、浮点数（实数）、十进制数
- 非数值数据：逻辑数（包括位串）、西文字符和汉字

◆ 计算机内部所有信息都用二进制（即：0和1）进行编码

◆ 用二进制编码的原因：

- 制造二个稳定态的物理器件容易
- 二进制编码、计数、运算规则简单
- 正好与逻辑命题对应，便于逻辑运算，并可方便地用逻辑电路实现算术运算

◆ 真值和机器数

- 机器数：用0和1编码的计算机内部的0/1序列
- 真值：机器数真正的值，即：现实中带正负号的数

◆ 不同数制之间的转换

2、数的机器码表示

- 原码表示：最高位为符号位，其余位为数值位。
- 补码表示：对其数值位各位按位取反末位加1符号位填1。
- 反码表示：一个负数的反码即为对其原码除符号位以外的各位按位取反，或补码减去末位的1。
- 移码表示：浮点数的指数都用移码编码的方式表示。

通常用于阶码表示。（定点整数）

移码的定义： $[X]_{\text{移}} = 2^{n-1} + X$, $2^{n-1} - 1 \geq X \geq -2^{n-1}$

3、科学计数法(Scientific Notation)与浮点数

Example:

mantissa (尾数) $\rightarrow$ 6.02 $\times$ 10²¹ $\leftarrow$ *exponent* (阶码、指数)
 $\nwarrow$ $\nearrow$
decimal point *radix* (base, 基)

- **Normalized form** (规格化形式) : 小数点前只有一位非0数
- 同一个数有多种表示形式。例：对于数 1/1,000,000,000
 - Normalized (唯一的规格化形式): 1.0×10^{-9}
 - Unnormalized (非规格化形式不唯一) : 0.1×10^{-8} , 10.0×10^{-10}

for Binary Numbers:

mantissa (尾数) $\rightarrow$ 0.101_{two} $\times$ 2⁻¹⁰ $\leftarrow$ *exponent* (指数)
 $\nwarrow$ $\nearrow$
binary point 基为2

只要对尾数和指数分别编码，就可表示一个浮点数（即：实数）

IEEE 754 Floating Point Standard

规格化数: $\pm 1.\text{xxxxxxxxxx}_{\text{two}} \times 2^{\text{Exponent}}$

Single Precision : (Double Precision is similar)

S	Exponent	Significand
1 bit	8 bits (11)	23 bits (52)

- Sign bit: 1 表示negative ; 0表示 positive
- Exponent (阶码 / 指数) : 全0和全1用来表示特殊值!
 - SP规格化数阶码范围为0000 0001 (-126) ~ 1111 1110 (127)
 - bias为 $2^{n-1}-1=127$ (single), $2^{n-1}-1=1023$ (double)
- Significand (尾数) :
 - 规格化尾数最高位总是1, 所以隐含表示, 省1位
 - 1 + 23 bits (single) , 1 + 52 bits (double)

SP: $(-1)^S \times (1 + \text{Significand}) \times 2^{(\text{Exponent}-127)}$

DP: $(-1)^S \times (1 + \text{Significand}) \times 2^{(\text{Exponent}-1023)}$

例1：若浮点数x的754标准存储格式为 $(41360000)_{16}$ ，求其浮点数的十进制数值。

解：将16进制数展开后，可得二进制数格式为

0 100 00010 011 0110 0000 0000 0000 0000

符号S 阶码E(8位) 尾数M(23位)

指数 $e = E - 127 = 10000010 -$

$01111111 = 00000011 = (3)_{10}$

包括隐藏位1的尾数

$1.M = 1.011\ 0110\ 0000\ 0000\ 0000\ 0000 = 1.011011$

于是有 $x = (-1)^S \times 1.M \times 2^e = +(1.011011) \times 2^3 = +1011.011 = (11.375)_{10}$

例2：将数 $(20.59375)_{10}$ 转换成754标准的32位浮点数的 二进制存储格式。

解：首先分别将整数和分数部分转换成二进制数：

$$20.59375 = 10100.10011$$

然后移动小数点，使其在第1，2位之间

$$10100.10011 = 1.010010011 \times 2^4$$

$e=4$ 于是得到：

$$S=0, E=4+127=131, M=010010011$$

最后得到32位浮点数的二进制存储格式为：

$$01000001101001001100000000000000 = (41A4C000)_{16}$$

4、数据的基本宽度

◆ “字” 和 “字长” 的概念不同

- “字长” 指定点运算数据通路的宽度。

（数据通路指CPU内部数据流经的路径以及路径上的部件，主要是CPU内部进行数据运算、存储和传送的部件，这些部件的宽度基本上要一致，才能相互匹配。因此，“字长”等于CPU内部总线的宽度、运算器的位数、通用寄存器的宽度等。）

- “字” 表示被处理信息的单位，用来度量数据类型的宽度。
- 字和字长的宽度可以一样，也可不同。

例如，x86体系结构定义“字”的宽度为16位，但从386开始字长就是32位了。

5、数据量的度量单位

- ◆ 存储二进制信息时的度量单位要比字节或字大得多
- ◆ 容量经常使用的单位有：
 - “千字节” (KB), $1\text{KB}=2^{10}\text{字节}=1024\text{B}$
 - “兆字节” (MB), $1\text{MB}=2^{20}\text{字节}=1024\text{KB}$
 - “千兆字节” (GB), $1\text{GB}=2^{30}\text{字节}=1024\text{MB}$
 - “兆兆字节” (TB), $1\text{TB}=2^{40}\text{字节}=1024\text{GB}$
- ◆ 通信中的带宽使用的单位有：
 - “千比特/秒” (kb/s), $1\text{kbps}=10^3\text{ b/s}=1000\text{ bps}$
 - “兆比特/秒” (Mb/s), $1\text{Mbps}=10^6\text{ b/s}=1000\text{ kbps}$
 - “千兆比特/秒” (Gb/s), $1\text{Gbps}=10^9\text{ b/s}=1000\text{ Mbps}$
 - “兆兆比特/秒” (Tb/s), $1\text{Tbps}=10^{12}\text{ b/s}=1000\text{ Gbps}$

如果把b换成B，则表示字节而不是比特（位）

例如，10MBps表示 10兆字节/秒

6、数据的存储和排列顺序

大端方式（**Big Endian**）：**MSB**所在的地址是数的地址

e.g. IBM 360/370, Motorola 68k, MIPS, Sparc, HP PA

小端方式（**Little Endian**）：**LSB**所在的地址是数的地址

e.g. Intel 80x86, DEC VAX

BIG Endian versus Little Endian

Ex1: Memory layout of a number ABCDH located in 1000

In Big Endian:	→	CD	1001
		AB	1000
In Little Endian:	→	AB	1001
		CD	1000

Ex2: Memory layout of a number 00ABCDEFH located in 1000

In Big Endian:	→	00	1000
		AB	1001
		CD	1002
		EF	1003
In Little Endian:	→	00	1003
		AB	1002
		CD	1001
		EF	1000

7、奇偶校验码

基本思想：增加一位奇（偶）校验位并一起存储或传送，根据终部件得到的相应数据和校验位，再求出新校验位，最后根据新校验位确定是否发生了错误。

实现原理：假设数据 $B=b_{n-1}b_{n-2}\dots b_1b_0$ 从源部件传送至终部件。在终部件接收到的数据为 $B'=b_{n-1}'b_{n-2}'\dots b_1'b_0'$ 。

第一步：在源部件求出奇（偶）校验位 P 。

若采用奇校验，则 $P=b_{n-1}\oplus b_{n-2}\oplus \dots\oplus b_1\oplus b_0\oplus 1$ 。

若采用偶校验，则 $P=b_{n-1}\oplus b_{n-2}\oplus \dots\oplus b_1\oplus b_0$ 。

第二步：在终部件求出奇（偶）校验位 P' 。

若采用奇校验，则 $P'=b_{n-1}'\oplus b_{n-2}'\oplus \dots\oplus b_1'\oplus b_0'\oplus 1$ 。

若采用偶校验，则 $P'=b_{n-1}'\oplus b_{n-2}'\oplus \dots\oplus b_1'\oplus b_0'$ 。

第三步：计算最终的校验位 P^* ，并根据其值判断有无奇偶错。

假定 P 在终部件接受到的值为 P'' ，则 $P^*=P'\oplus P''$

① 若 $P^*=1$ ，则表示终部件接受的数据有奇数位错。

② 若 $P^*=0$ ，则表示终部件接受的数据正确或有偶数个错。

运算方法和运算部件

1、计算机硬件的操作数放在什么地方？

◆CPU的寄存器（register） 中——寄存器操作数

寄存器是建造计算机的基石，因为它们是硬件设计中用到的基本单元，对于程序员也是可见的，但其数量有限。

◆存储器中——存储器操作数

复杂数据结构，如数组、结构体等包含大量数据元素，不可能映射到数量有限的寄存器上，只能存储到存储器中。

◆指令中——立即数

有些操作数直接与指令存放在一起，称为立即数，而MIPS中专门设置有一些立即数指令，如addi, slti等。

2、基本运算部件

一般情况下，用一个专门的算术逻辑部件（ALU）来完成基本逻辑运算和定点数加减运算。

各类定点乘除运算和浮点数运算则可利用加法器或者ALU和移位器来实现。

因此，基本运算部件是加法器、ALU和移位器，**ALU的核心部件是加法器。**

3、n位整数加/减运算器

- 补码加减运算公式

$$[A+B]_{\text{补}} = [A]_{\text{补}} + [B]_{\text{补}} \pmod{2^n}$$

$$[A-B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \pmod{2^n}$$

— 实现减法的主要工作在于：求 $[-B]_{\text{补}}$

问题：如何求 $[-B]_{\text{补}}$ ？

$$[-B]_{\text{补}} = \overline{[B]_{\text{补}}} + 1$$

溢出概念与检测方法

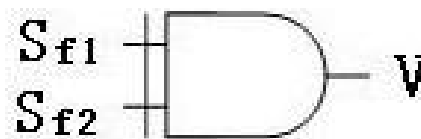
- 溢出的概念
 - 数的表示范围超过了机器所规定的范围称为“溢出”。
 - 可能产生溢出的情况
 - 两正数加，变负数，正溢（大于机器所能表示的最大数）
 - 两负数加，变正数，负溢（小于机器所能表示的最小数）

检测方法

双符号位法（变形补码） $[x]_{\text{补}} = 2^{n+2} + x$
(mod 2^{n+2})

S_{f1} S_{f2}

0	0	正确（正数）
0	1	正溢
1	0	负溢
1	1	正确（负数）

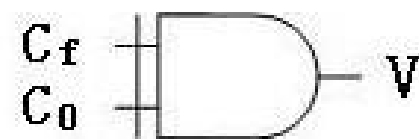


S_{f1} 表示正确的符号，逻辑表达式为 $V = S_{f1} \oplus S_{f2}$ ，
可以用异或门来实现

单符号位法

- C_f C_0

0	0	正确 (正数)
0	1	正溢
1	0	负溢
1	1	正确 (负数)



- $V = C_f \oplus C_0$, 其中 C_f 为符号位产生的进位, C_0 为最高有效位产生

4、浮点数加减法

(假定: X_m 、 Y_m 分别是X和Y的尾数, X_e 和 Y_e 分别是X和Y的阶码)

- (1) 求阶差: $\Delta e = Y_e - X_e$ (若 $Y_e > X_e$, 则结果的阶码为 Y_e)
- (2) 对阶: 将 X_m 右移 Δe 位, 尾数变为 $X_m * 2^{X_e - Y_e}$ (保留右移部分**附加位**)
- (3) 尾数加减: $X_m * 2^{X_e - Y_e} \pm Y_m$

(4) 规格化:

当尾数高位为0, 则需左规: 尾数左移一次, 阶码减1, 直到MSB为1或阶码为00000000 (-126, 非规格化数)

每次阶码减1后要判断阶码是否下溢 (比最小可表示的阶码还要小)

当尾数最高位有进位, 需右规: 尾数右移一次, 阶码加1, 直到MSB为1

每次阶码加1后要判断阶码是否上溢 (比最大可表示的阶码还要大)

阶码溢出异常处理: 阶码上溢, 则结果溢出; 阶码下溢到无法用非规格化数表示, 则结果为0

(5) 如果尾数比规定位数长 (有附加位), 则需考虑舍入 (有多种舍入方式)

(6) 若**运算结果尾数**是0, 则需要将阶码也置0。为什么?

尾数为0说明结果应该为0 (阶码和尾数为全0)。

0.00...0001 $\times 2^{-126}$

浮点数加法运算举例

例：用二进制浮点数形式计算 $0.5 + (-0.4375) = ?$

$$0.4375 = 0.25 + 0.125 + 0.0625 = 0.0111\text{B}$$

解： $0.5 = 1.000 \times 2^{-1}$, $-0.4375 = -1.110 \times 2^{-2}$

对 阶： $-1.110 \times 2^{-2} \rightarrow -0.111 \times 2^{-1}$

加 减： $1.000 \times 2^{-1} + (-0.111 \times 2^{-1}) = 0.001 \times 2^{-1}$

左 规： $0.001 \times 2^{-1} \rightarrow 1.000 \times 2^{-4}$

判溢出： 无

结果为： $1.000 \times 2^{-4} = 0.0001000 = 1/16 = 0.0625$

IEEE 754 浮点数加法运算举例

已知 $x=0.5$, $y=-0.4375$, 求 $x+y=?$ (用IEEE754标准单精度格式计算)

解: $x=0.5=1/2=(0.100...0)_2=(1.00...0)_2 \times 2^{-1}$

$y=-0.4325=(-0.01110...0)_2=(-1.110..0)_2 \times 2^{-2}$

$[x]_{\text{浮}}=0\ 01111110,00...0$ $[y]_{\text{浮}}=1\ 01111101,110...0$

对阶: $[\Delta E]_{\text{补}}=0111\ 1110 + 1000\ 0011=0000\ 0001$, $\Delta E=1$

故对 y 进行对阶: $[y]_{\text{浮}}=1\ 0111\ 1110\ 1110...0$ (高位补隐藏位)

尾数相加: $01.0000...0 + (10.1110...0) = 00.00100...0$

(原码加法, 最左边一位为符号位, 符号位分开处理)

左规: $+(0.00100...0)_2 \times 2^{-1} = +(1.00...0)_2 \times 2^{-4}$

(阶码减3, 实际上是加了三次11111111)

$[x+y]_{\text{浮}}=0\ 0111\ 1011\ 00...0$

$x+y=(1.0)_2 \times 2^{-4}=1/16=0.0625$

5、内部总线

- **内部总线**

- 机器内部各部份数据传送频繁,可以把寄存器间的数据传送通路加以归并,组成总线结构。
- 分类
 - 所处位置
 - 内部总线 (CPU内)
 - 外部总线 (系统总线)
 - 逻辑结构
 - 单向传送总线
 - 双向传送总线

第三章 存储系统

1、基本术语

- 记忆单元（存储基元 / 存储元 / 位元）（Cell）
 - 具有两种稳态的能够表示二进制数码0和1的物理器件
- 存储单元 / 编址单位（Addressing Unit）
 - 具有相同地址的位构成一个存储单元，也称为一个编址单位
- 存储体 / 存储矩阵 / 存储阵列（Bank）
 - 所有存储单元构成一个存储阵列
- 编址方式（Addressing Mode）
 - 字节编址、按字编址
- 存储器地址寄存器（Memory Address Register - MAR）
 - 用于存放主存单元地址的寄存器
- 存储器数据寄存器（Memory Data Register-MDR (或MBR)）
 - 用于存放主存单元中的数据的寄存器

2、存储器分类

按功能/容量/速度/所在位置分类

- 寄存器(Register)
 - 封装在CPU内, 用于存放当前正在执行的指令和使用的数据
 - 用触发器实现, 速度快, 容量小 (几~几十个)
- 高速缓存(Cache)
 - 位于CPU内部或附近, 用来存放当前要执行的局部程序段和数据
 - 用SRAM实现, 速度可与CPU匹配, 容量小 (几MB)
- 内存存储器MM (主存储器Main (Primary) Memory)
 - 位于CPU之外, 用来存放已被启动的程序及所用的数据
 - 用DRAM实现, 速度较快, 容量较大 (几GB)
- 外存储器AM (辅助存储器Auxiliary / Secondary Storage)
 - 位于主机之外, 用来存放暂不运行的程序、数据或存档文件
 - 用磁表面或光存储器实现, 容量大而速度慢

3、存储容量

存储器芯片上能存储的二进制数的位数或者所包含的字节总数。

计算机内，信息的最小表示单位是：“位 (bit)”。

CPU访问存储器的最小单位是：“字节 (Byte)”。

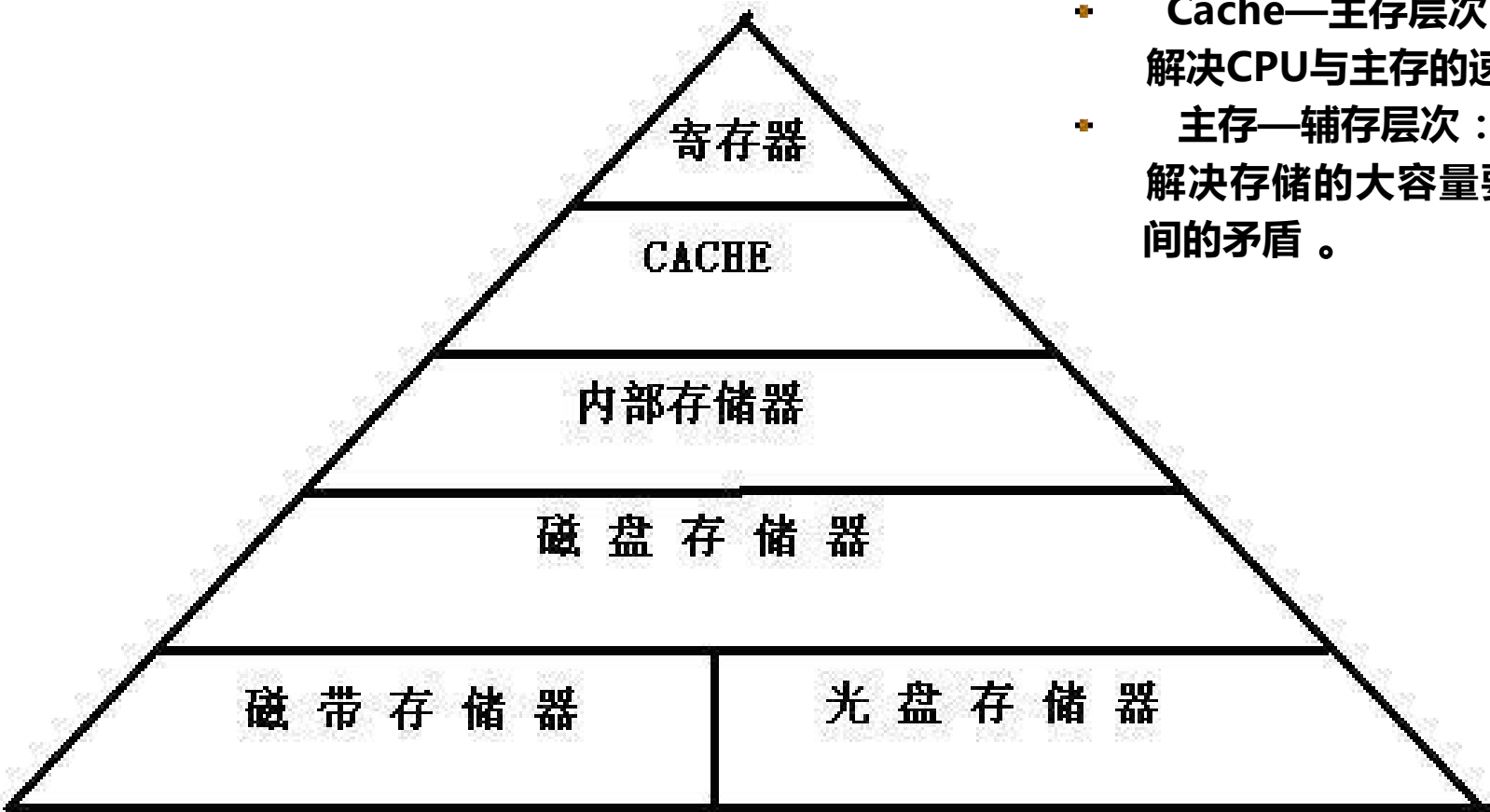
存储器芯片容量取决于存储单元的个数和每个单元包含的位数。

存储器容量 (S) = 存储单元数 (p) × 数据位数 (i)

4、存储器芯片的扩展

- 字扩展（位数不变、扩充容量）
 - 地址线、读/写控制线等对应相接，片选信号连译码输出
- 位扩展（字数不变，位数扩展）
 - 芯片的地址线及读/写控制线对应相接，而数据线单独引出
- 字位同时扩展（字和位同时扩展）
 - 地址线、读/写控制线等对应相接，片选信号则分别与外部译码器各个译码输出端相连

5、存储器的层次化结构



- Cache—主存层次：
解决CPU与主存的速度上的差距；
- 主存—辅存层次：
解决存储的大容量要求和低成本之间的矛盾。

核心是解决容量、速度、价格间的矛盾，建立起多层存储结构。

一个金字塔结构的多层存储体系充分体现出容量和速度关系

6、Cache

◦ 大量典型程序的运行情况分析结果表明

- 在较短时间间隔内，程序产生的地址往往集中在一个很小范围内

这种现象称为程序访问的局部性：**空间局部性、时间局部性**

空间局部性：被访问的某个存储单元的邻近单元在一个较短的时间间隔内很可能也被访问。

时间局部性：被访问的某个存储单元在一个较短的时间间隔内很可能又被访问

◦ 程序具有访问局部性特征的原因

- **指令：**指令按序存放，地址连续，循环程序段或子程序段重复执行
- **数据：**连续存放，数组元素重复、按序访问

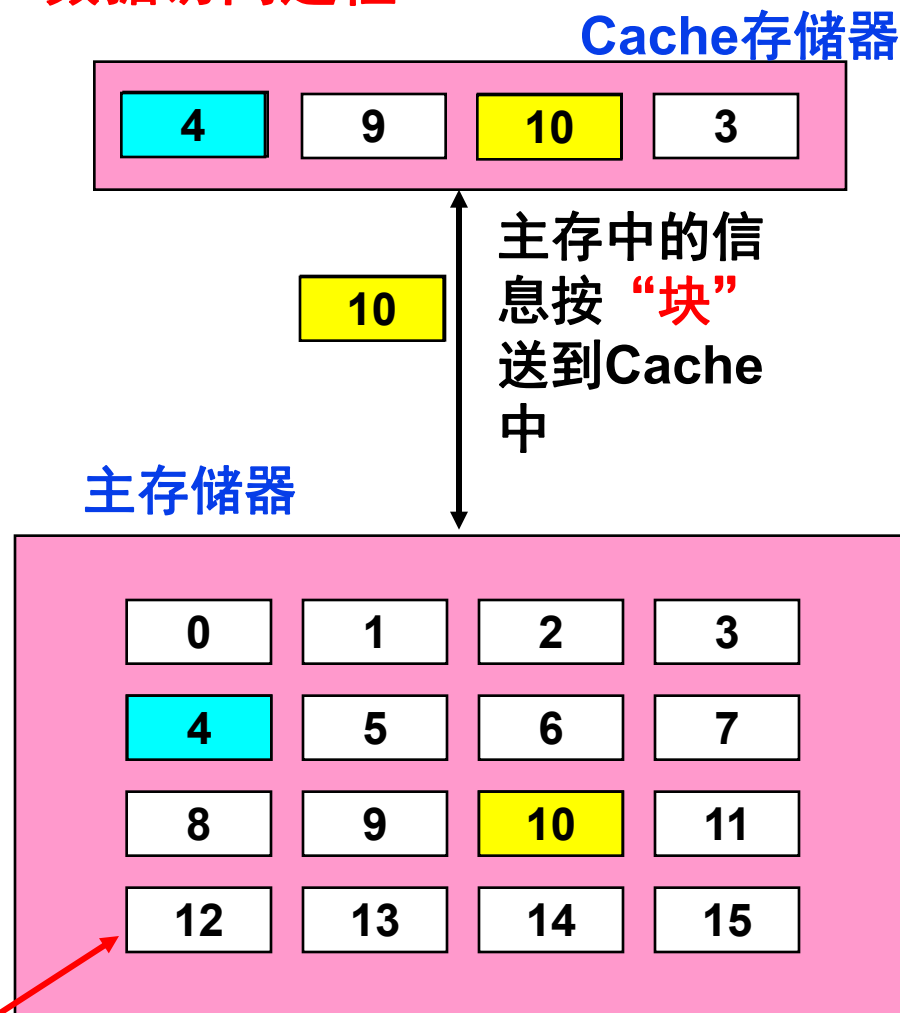
◦ 为什么引入Cache会加快访存速度？

- 在CPU和主存之间设置一个快速小容量的存储器，其中总是存放最活跃（被频繁访问）的程序和数据，由于程序访问的局部性特征，大多数情况下，CPU能直接从这个高速缓存中取得指令和数据，而不必访问主存。
高速缓存就是位于主存和CPU之间的Cache！

Cache(高速缓存)是什么样的？

- Cache是一种小容量高速缓冲存储器，它由SRAM组成。
- Cache直接制作在CPU芯片内，速度几乎与CPU一样快。
- 程序运行时，CPU使用的一部分数据/指令会预先成批拷贝在Cache中，Cache的内容是主存储器中部分内容的映象。
- 当CPU需要从内存读(写)数据或指令时，先检查Cache，若有，就直接从Cache中读取，而不用访问主存储器。

数据访问过程：



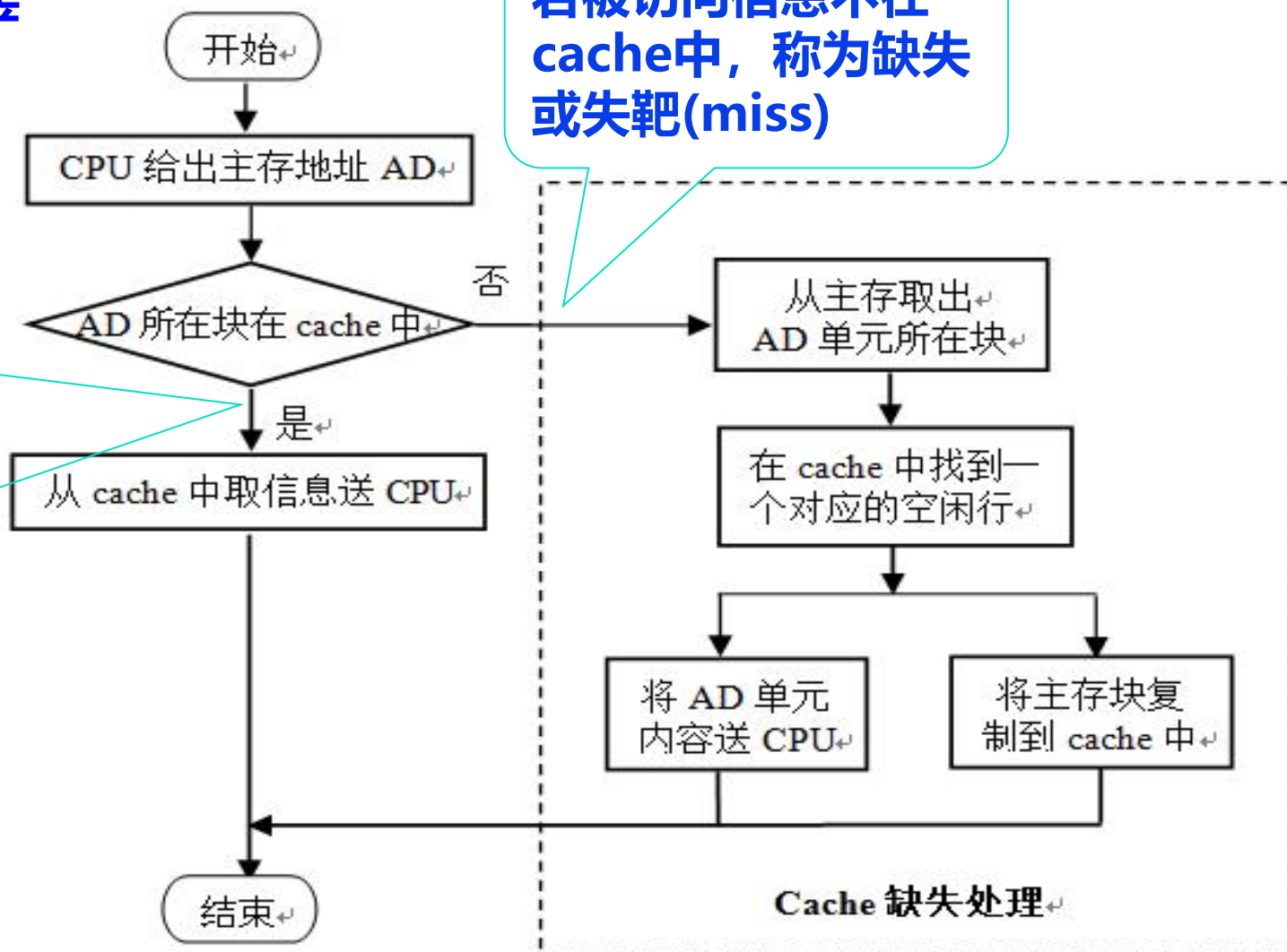
Cache 的操作过程

问题：什么情况下，CPU产生访存要求？

执行指令时！

若被访问信息在 cache 中，称为命中 (hit)

若被访问信息不在 cache 中，称为缺失或失靶 (miss)



7、Cache映射(Cache Mapping)

◦ 什么是Cache的映射功能?

- 把访问的局部主存区域取到Cache中时, 该放到Cache的何处?
- Cache行比主存块少, 多个主存块映射到一个Cache行中

◦ 如何进行映射?

- 把主存空间划分成大小相等的主存块 (Block)
- Cache中存放一个主存块的对应单位称为槽 (Slot) 或行 (line)
有书中也称之为块 (Block) , 有书称之为页 (page) (不妥!)
- 将主存块和Cache行按照以下三种方式进行映射
 - 直接(Direct): 每个主存块映射到Cache的固定行
 - 全相联(Full Associate): 每个主存块映射到Cache的任一行
 - 组相联(Set Associate): 每个主存块映射到Cache固定组中任一行

8、替换(Replacement)算法

当一个新的主存块需要拷贝到cache，而允许存放此块的行位置都被其他主存块占满时，就要产生替换。

替换算法有以下几种：

- (1) **FIFO** (先进先出算法)
- (2) **LRU** (近期最少使用算法)：LRU算法将近期内长久未被访问过的行换出。
- (3) **LFU** (最不经常使用算法)：LFU算法认为应将一段时间内被访问次数最少的那行数据换出。
- (4) **RAND** (随机替换)

例：在Cache存储器组织中，如果采用LRU替换算法，假设Cache组内有5个信息块，原始状态已装入1、2、3、4、5并依次运行过，如果再要运行的块号依次为2、6、3、7、5、4则将先后有那些块_____被替换掉。

最近使用的块	块调入	顺序	2	6	3	7	5	4
		5	2	6	3	7	5	4
		4	5	2	6	3	7	5
		3	4	5	2	6	3	7
		2	3	4	5	2	6	3
		1	1	3	4	5	2	6
较早使用的块	被替换出块			1		4		2

被替换出的块依次为：1、4、2

第四章 指令系统

1 指令系统

- **指令**：就是要计算机执行某种操作的命令。从计算机组成的层次结构来说，计算机的指令有**微指令**、**机器指令**和**宏指令**之分。微指令是微程序级的命令，它属于硬件；
- **宏指令**：由若干条机器指令组成的软件指令，它属于软件；
- **机器指令**：介于微指令与宏指令之间，通常简称为指令，每一条指令可完成一个独立的算术运算或逻辑运算操作。
- 一台计算机中所有机器指令的集合，称为这台计算机的**指令系统**。

一条指令中应该有几个地址码字段？

零地址指令

- (1) 无需操作数 如：空操作 / 停机等
- (2) 所需操作数为默认的 如：堆栈 / 累加器等

形式：

OP

一地址指令

其地址既是操作数的地址，也是结果的地址

- (1) 单目运算：如：取反 / 取负等
- (2) 双目运算：另一操作数为默认的 如：累加器等

形式：

OP	A1
----	----

二地址指令（最常用）

分别存放双目运算中两个操作数，并将其中一个地址作为结果的地址。

形式：

OP	A1	A2
----	----	----

三地址指令（RISC风格）

分别作为双目运算中两个源操作数的地址和一个结果的地址。

形式：

OP	A1	A2	A3
----	----	----	----

多地址指令

用于成批数据处理的指令，如：向量 / 矩阵等运算的SIMD指令。

2、操作数类型和存储方式

操作数是指令处理的对象，与高级语言数据类型对应，基本类型有哪些？

地址（指针）

被看成无符号整数，用来参加运算以确定主(虚)存地址

数值数据

定点数(整数)：一般用二进制补码表示

浮点数(实数)：大多数机器采用IEEE754标准

十进制数：用NBCD码表示，压缩/非压缩（汇编程序设计时用）

位、位串、字符和字符串

用来表示文本、声音和图像等

» 4 bits is a nibble（一个十六进制数字）

» 8 bits is a byte

» 16 bits is a half-word

» 32 bits is a word

存放在寄存器
或内存单元中

逻辑(布尔)数据

按位操作（0-假 / 1-真）

3、寻址方式 (Addressing Modes)

- ◆ 什么是“寻址方式”？

指令或操作数地址的指定方式。即：根据地址找到指令或操作数的方法。

- ◆ 指令的寻址----简单

正常：PC增值

跳转 (jump / branch / call / return)：同操作数的寻址

- ◆ 操作数的寻址----复杂

操作数来源：寄存器 / 外设端口 / 主(虚)存 / 栈顶

操作数结构：位 / 字节 / 半字 / 字 / 双字 / 一维表 / 二维表 / ...

通常寻址方式特指 “操作数的寻址”

- ◆ 基本寻址方式

立即 / 直接 / 间接 / 寄存器 / 寄存器间接 / 偏移 / 堆栈

基本寻址方式的算法和优缺点

方式	算法	优点	缺点
隐含寻址	操作数在专用寄存器中	无存储器访问	数据范围 有限
立即寻址	操作数=A	无存储器访问	数据范围幅值有限
寄存器寻址	EA=R	无存储器访问	地址范围有限
直接寻址	EA=A	简单	地址范围有限
间接寻址	EA= (A)	大的地址范围	多重存储器访问
寄存器间接寻址	EA= (R)	大的地址范围	额外存储器访问
偏移寻址	EA=A+ (R)	灵活	复杂
段寻址	EA=A+ (R)	灵活	复杂
堆栈寻址	EA=栈顶	无需给出存储器地址	需要堆栈指示器

问题：以上各种寻址方式下，操作数在寄存器中还是在存储器中？

4、指令的分类

1.数据传送指令

3.逻辑运算指令

5.输入输出指令

7. 特权指令

2.算术运算指令

4.程序控制指令

6.字符串处理指令

8.其它指令

5、指令设计风格 – 按指令格式的复杂度来分

按指令格式的复杂度来分，有两种类型计算机：

复杂指令集计算机CISC (Complex Instruction Set Computer)

精简指令集计算机RISC (Reduce Instruction Set Computer)

早期CISC设计风格的主要特点

(1) 指令系统复杂

变长操作码 / 变长指令字 / 指令多 / 寻址方式多 / 指令格式多

(2) 指令周期长

绝大多数指令需要多个时钟周期才能完成

(3) 各种指令都能访问存储器

除了专门的存储器读写指令外，运算指令也能访问存储器

(4) 采用微程序控制

(5) 有专用寄存器

(6) 难以进行编译优化来生成高效目标代码

复杂指令集计算机CISC

◆ CISC的缺陷

- 日趋庞大的指令系统不但使计算机的研制周期变长，而且难以保证设计的正确性，难以调试和维护，并且因指令操作复杂而增加机器周期，从而降低了系统性能。

◆ 1975年IBM公司开始研究指令系统的合理性问题，John Cocks提出精简指令系统计算机 RISC (Reduce Instruction Set Computer)。

◆ 对CISC进行测试，发现一个事实：

- 在程序中各种指令出现的频率悬殊很大，最常使用的是一些简单指令，这些指令占程序的80%，但只占指令系统的20%。而且在微程序控制的计算机中，占指令总数20%的复杂指令占用了控制存储器容量的80%。

◆ 1982年美国加州伯克利大学的RISC I，斯坦福大学的MIPS，IBM公司的IBM801相继宣告完成，这些机器被称为第一代RISC机。

RISC设计风格的主要特点

(1) 简化的指令系统

指令少 / 寻址方式少 / 指令格式少 / 指令长度一致

(2) 以RR方式工作

除Load/Store指令可访问存储器外，其余指令都只访问寄存器。

(3) 指令周期短

以流水线方式工作，因而除Load/Store指令外，其他简单指令都只需一个或一个不到的时钟周期就可完成。

(4) 采用大量通用寄存器，以减少访存次数

(5) 采用组合逻辑电路控制，不用或少用微程序控制

(6) 采用优化的编译系统，力求有效地支持高级语言程序

ARM是典型的RISC处理器，82年以来新的指令集大多采用RISC体系结构
x86因为“兼容”的需要，保留了CISC的风格，同时也借鉴了RISC思想

6、ARM指令字段含义

例：将ARM汇编语言翻译成机器语言。已知5条ARM指令格式译码如下表所示：

指令名称	cond	F	I	opcode	S	R _e	R _d	operand2
ADD (加)	14	0	0	4	0	reg	reg	reg
SUB (减)	14	0	0	2	0	reg	reg	reg
ADD (立即数加)	14	0	1	4	0	reg	reg	constand (12 位)
LDR (取字)	14	1	—	24	—	reg	reg	address (12 位)
STR (存字)	14	1	—	25	—	reg	reg	address (12 位)

设r3寄存器中保存数组A的基值，h放在寄存器r2中。C语言程序语句 $A[30]=h+A[30]$ 可编译成如下3条汇编语句指令：

LDR r5 , [r3, #120]	;寄存器r5中获得A[30]
ADD r5 , r2 , r5	;寄存器r5中获得h+A[30]
STR r5 , [r3, #120]	;将h+A[30]存入到A[30]

请问这3条汇编语言指令的机器语言是什么？

解：首先利用十进制数来表示机器语言指令，然后转换成二进制机器指令。从表中我们可以确定3条机器语言指令：

LDR指令在第3字段（opcond）用操作码24确定。基值寄存器3指定在 第4字段（R_n），目的寄存器5指定在第6字段（R_d），选择A[30]（120 = 30×4）的offset字段放在最后一个字段（offset12）。

ADD指令在第4字段（opcode）用操作码4确定。3个寄存器操作（2、 5和5）分别被指定在第6、7、8字段。

STR指令在第3字段用操作码25确定，其余部分与LDR指令相同。

	cond	F	opcode			Rn	Rd	offset12
			I	opcode	S			operand2
十进制	14	1	24			3	5	120
	14	0	0	4	0	2	5	5
	14	1	25			3	5	120
二进制	1110	1	11000			0011	0101	0000 1111 0000
	1110	0	0	100	0	0010	0101	0000 0000 0101
	1110	1	11001			0011	0101	0000 1111 0000

指令名称	cond	F	I	opcode	S	R _e	R _d	operand2
ADD (加)	14	0	0	4	0	reg	reg	reg
SUB (减)	14	0	0	2	0	reg	reg	reg
ADD (立即数加)	14	0	1	4	0	reg	reg	constand (12 位)
LDR (取字)	14	1	—	24	—	reg	reg	address (12 位)
STR (存字)	14	1	—	25	—	reg	reg	address (12 位)

LDR r5 , [r3, #120]

ADD r5 , r2 , r5

STR r5 , [r3, #120]

	cond	F	opcode			Rn	Rd	offset12
			I	opcode	S			operand2
十进制	14	1	24			3	5	120
	14	0	0	4	0	2	5	5
	14	1	25			3	5	120
二进制	1110	1	11000			0011	0101	0000 1111 0000
	1110	0	0	100	0	0010	0101	0000 0000 0101
	1110	1	11001			0011	0101	0000 1111 0000

7、栈(Stack)的概念

◆ 栈的基本概念

- 是一个“先进后出”队列
- 需一个栈指针指向栈顶元素
- 每个元素长度一致
- 用“入栈”（push）和“出栈”（pop）操作访问栈元素

第五章 中央处理器

1、CPU的基本组成

CPU 的基本组成：运算器、cache 和控制器三大部分。

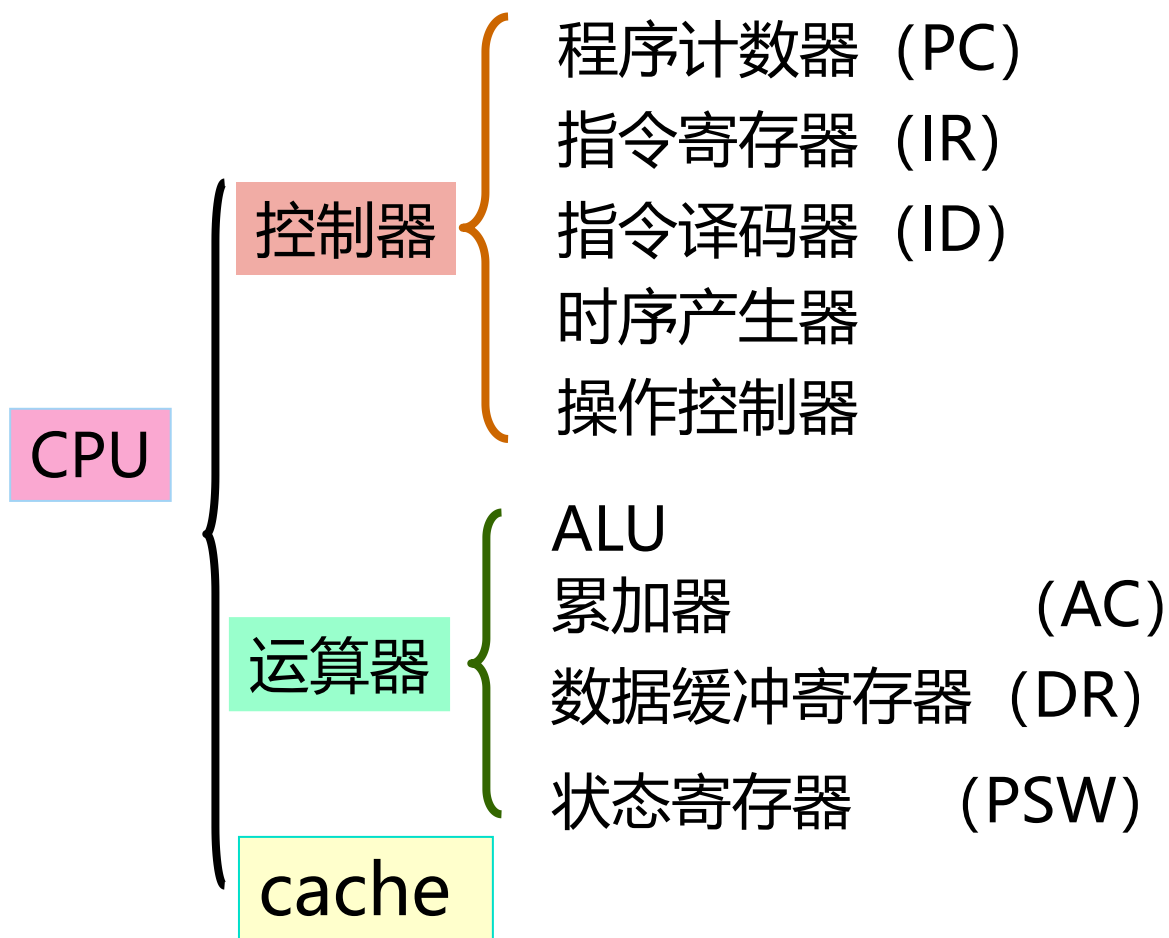
1. 控制器的主要功能：

- (1) 从内存中取出一条指令，并指出下一条指令在内存中的位置；
- (2) 对指令进行译码或测试，并产生相应的操作控制信号，以便启动规定的动作；
- (3) 指挥并控制CPU、内存和输入/输出设备之间数据流动的方向。

2. 运算器的主要功能：

- (1) 执行所有的算术运算；
- (2) 执行所有的逻辑运算，并进行逻辑测试。

CPU 的组成图

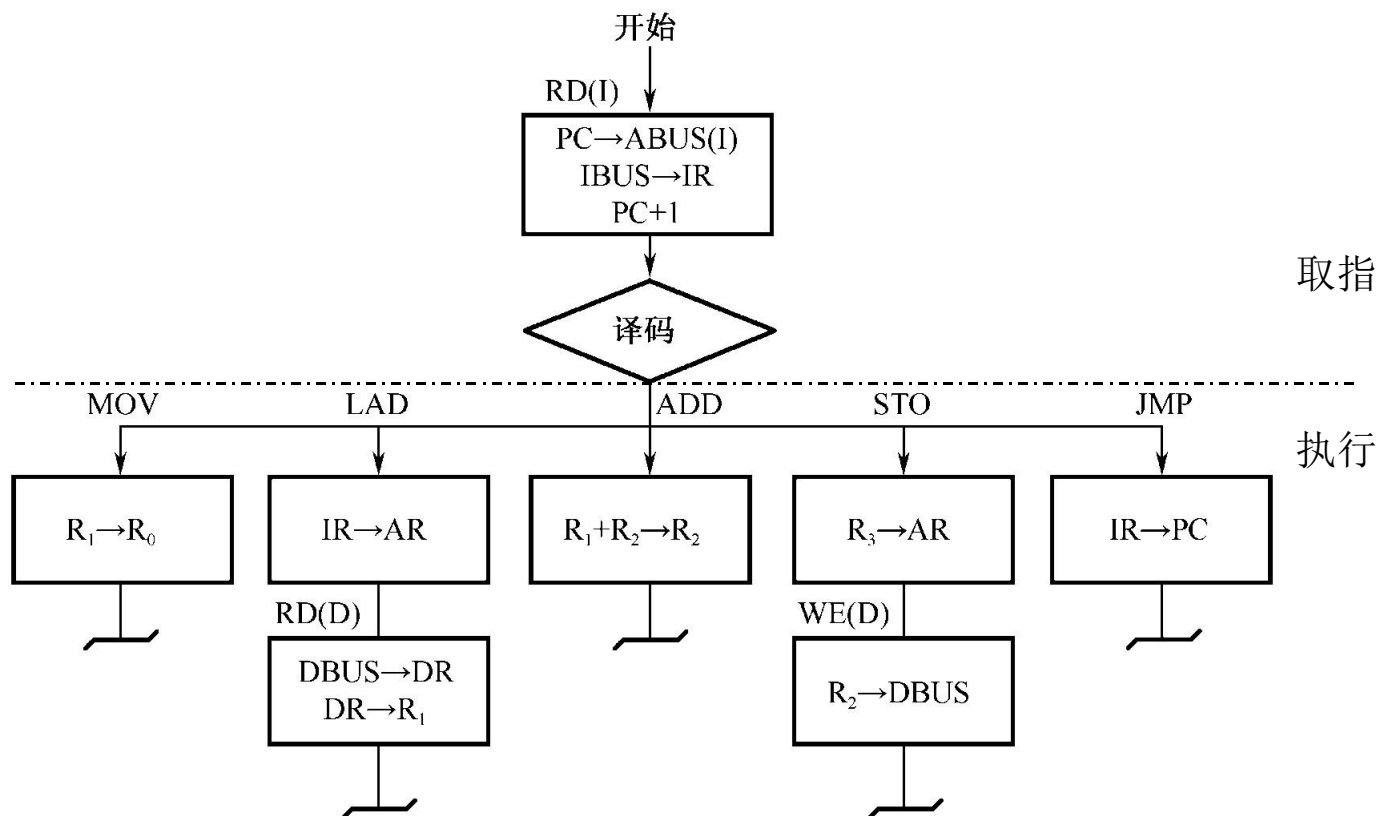


2、指令周期的基本概念

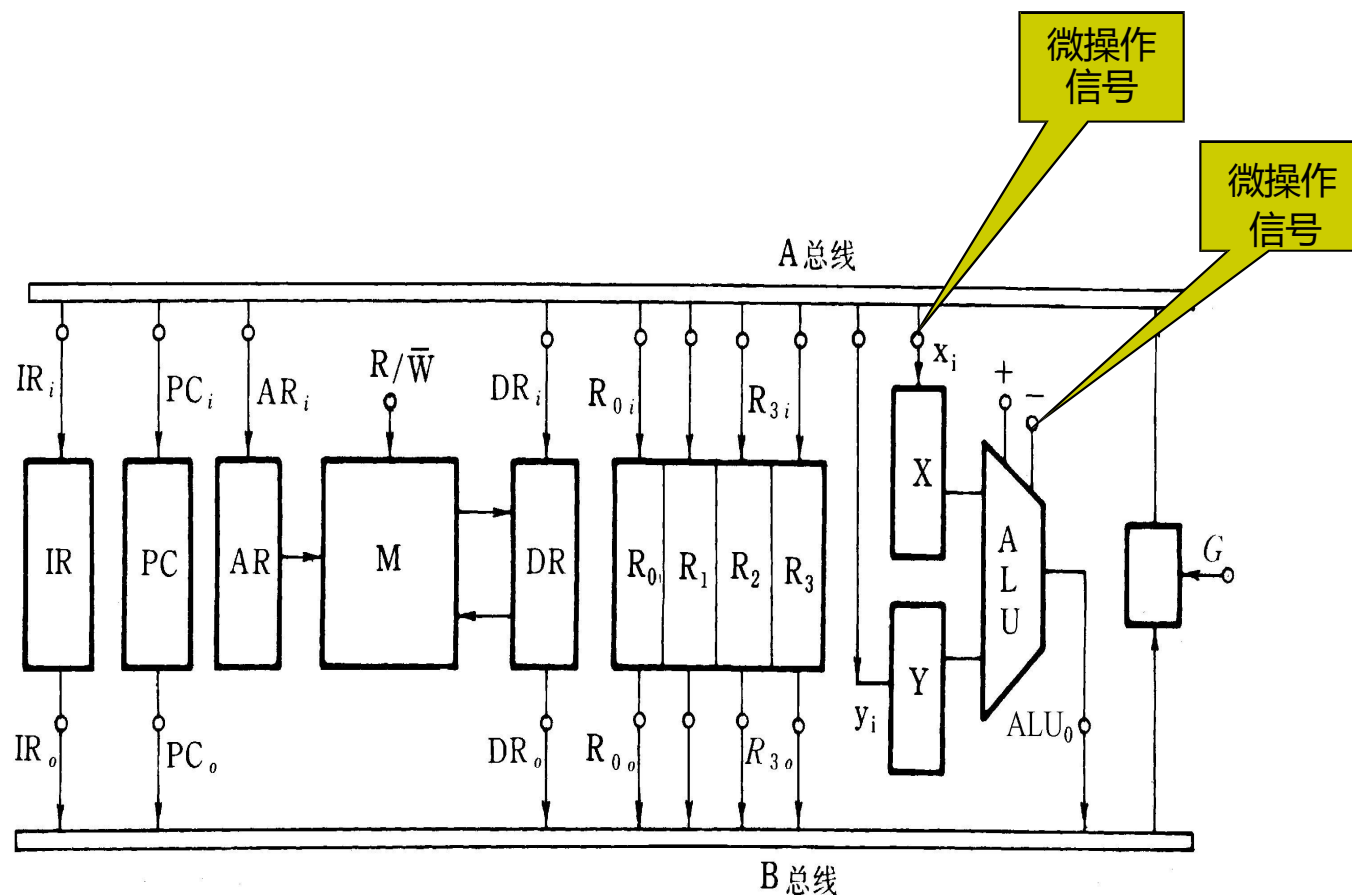
- 概念

- **指令周期**：指取指令、分析指令到执行完该指令所需的全部 时间。
 - 各种指令的指令周期相同吗？为什么？
- **CPU周期**通常又称**时钟周期**
 - 通常把一条指令周期划分为若干个机器周期，每个机器周期 完成一个基本操作。
 - 主存的工作周期(存取周期)为基础来规定CPU周期，比如， 可以用**CPU读取一个指令字的最短时间**来规定**CPU周期**
 - 不同的指令，可能包含不同数目的CPU周期。
 - 一个CPU周期中，包含若干个节拍脉冲（T周期）。
 - 单周期、多周期的概念

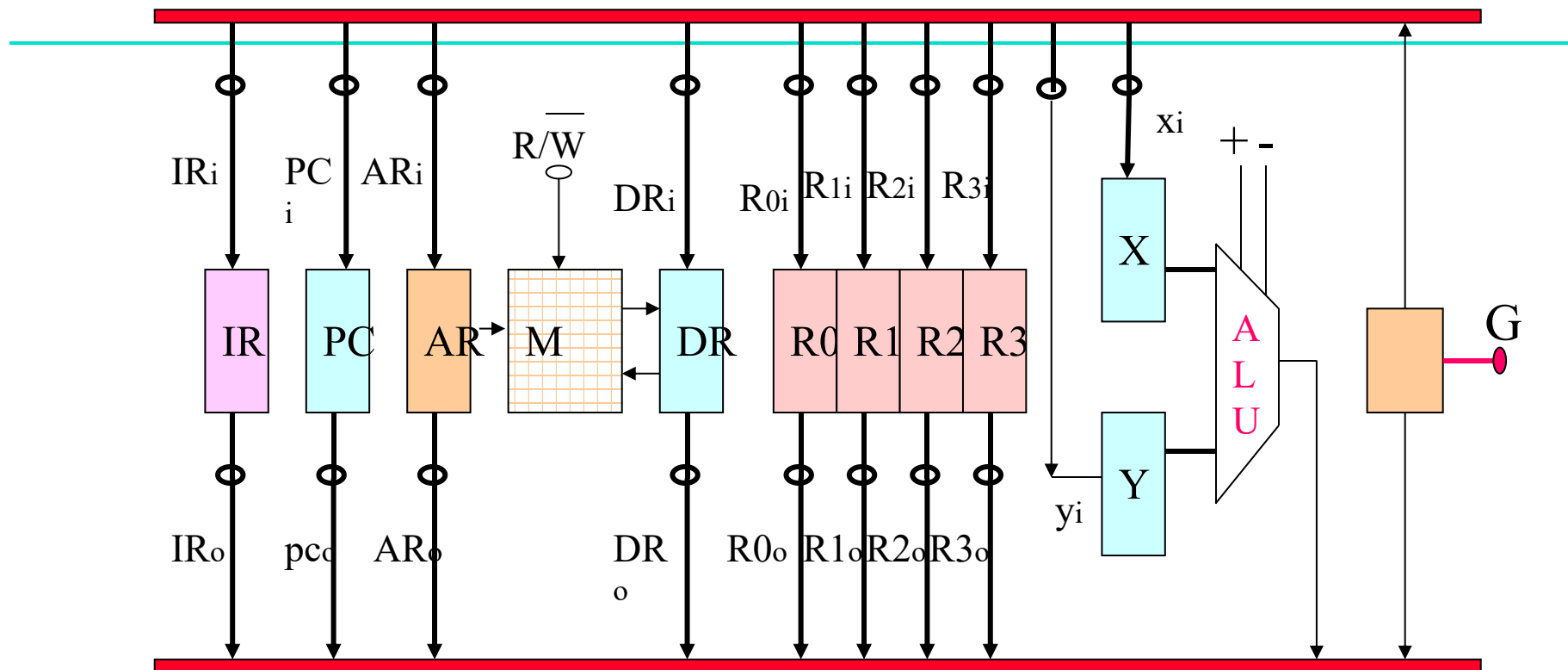
用方框图语言表示指令周期



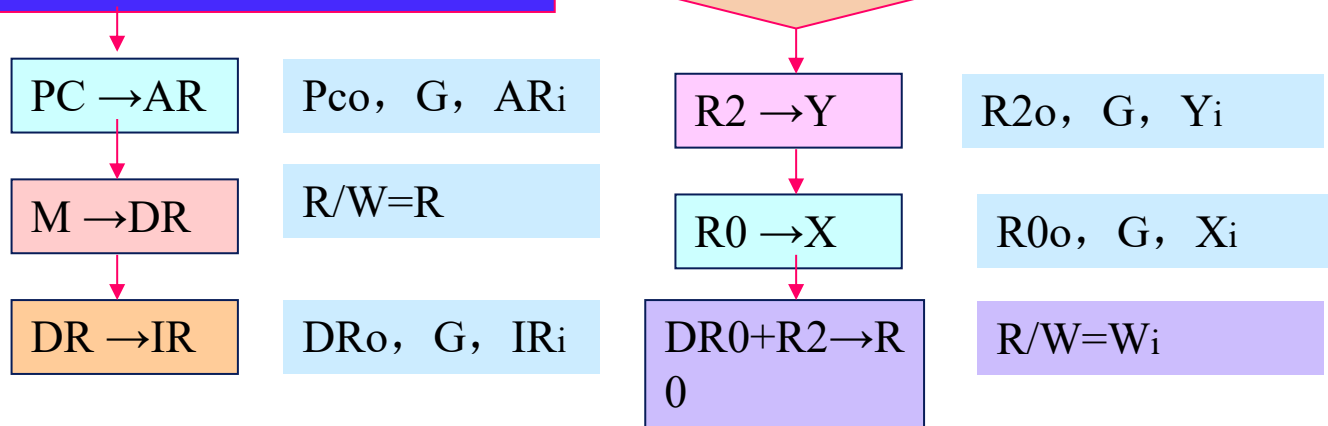
双总线结构机器的数据通路图



- (1) ADD R2,R0 画出其指令周期流程图, 并列出相应的微操作控制信号序列
 (2) SUB R1,R3 画出其指令周期流程图, 并列出相应的微操作控制信号序列



ADD R2 , R0 指令周期流程图



3、微程序控制器

微命令：控制部件向执行部件发出的各种控制命令叫作微命令，它是构成控制序列的最小单位。

微操作：是微命令的操作过程。

- 微命令和微操作是一一对应的。
- 微命令是微操作的控制信号，微操作是微命令的操作过程。
- 微操作是执行部件中最基本的操作。

互斥的微操作，是指不能同时或不能在同一个节拍内并行执行的微操作。

相容的微操作，是指能够同时或在同一个节拍内并行执行的微操作。

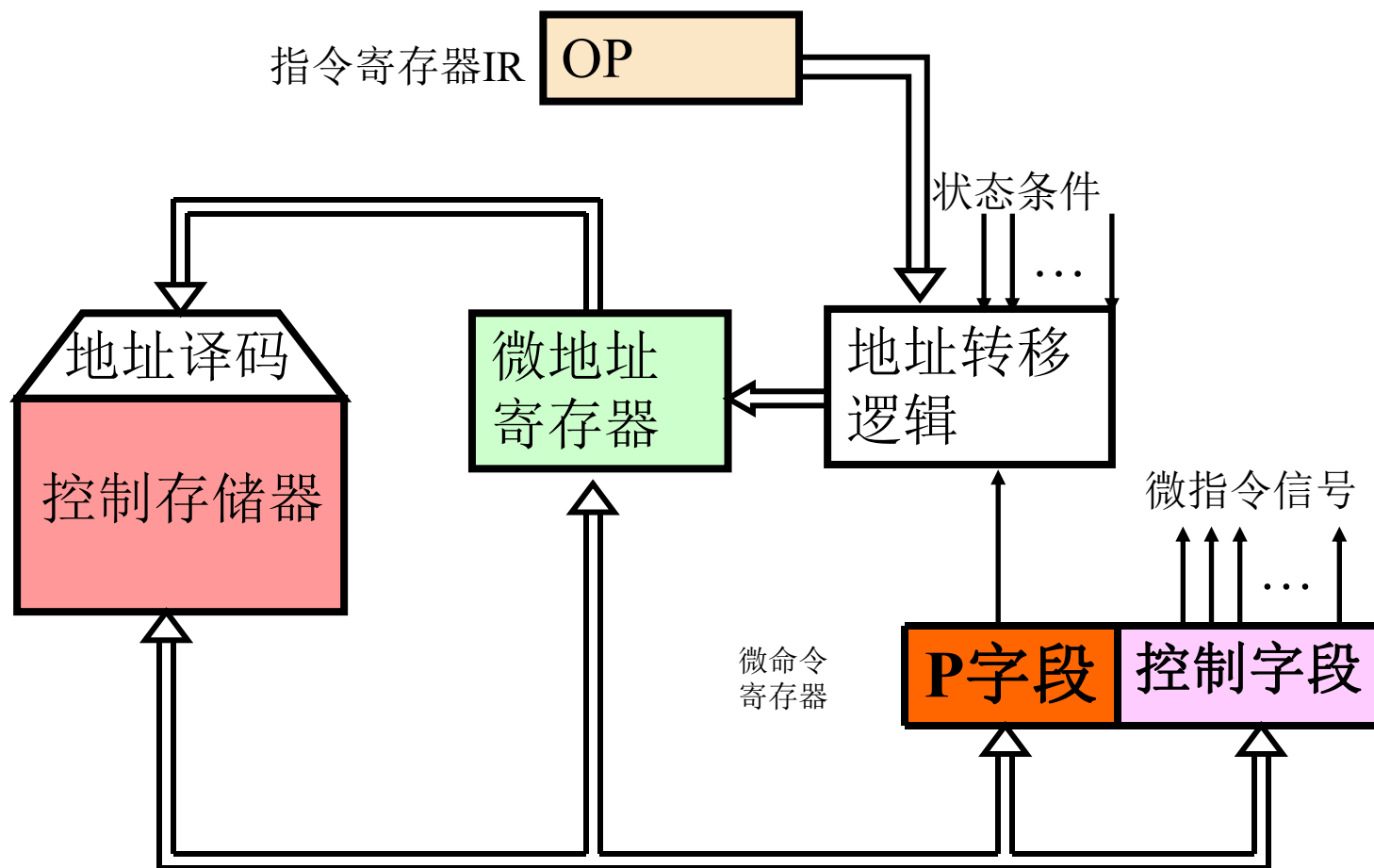
微指令：把在同一CPU周期内并行执行的微操作控制信息，存储在控制存储器里，称为一条微指令（Microinstruction）。

- 它是微命令的组合，微指令存储在控制器中的控制存储器中
- 一条微指令通常至少包含两大部分信息：
 - **操作控制字段**，又称微操作码字段，用以产生某一步操作所需的各个微操作控制信号。
 - **顺序控制字段**，又称微地址码字段，用以控制产生下一条要执行的微指令地址。

微程序

- 一系列微指令的有序集合就是微程序。
 - 一段微程序对应一条机器指令。
 - 微地址：存放微指令的控制存储器的单元地址

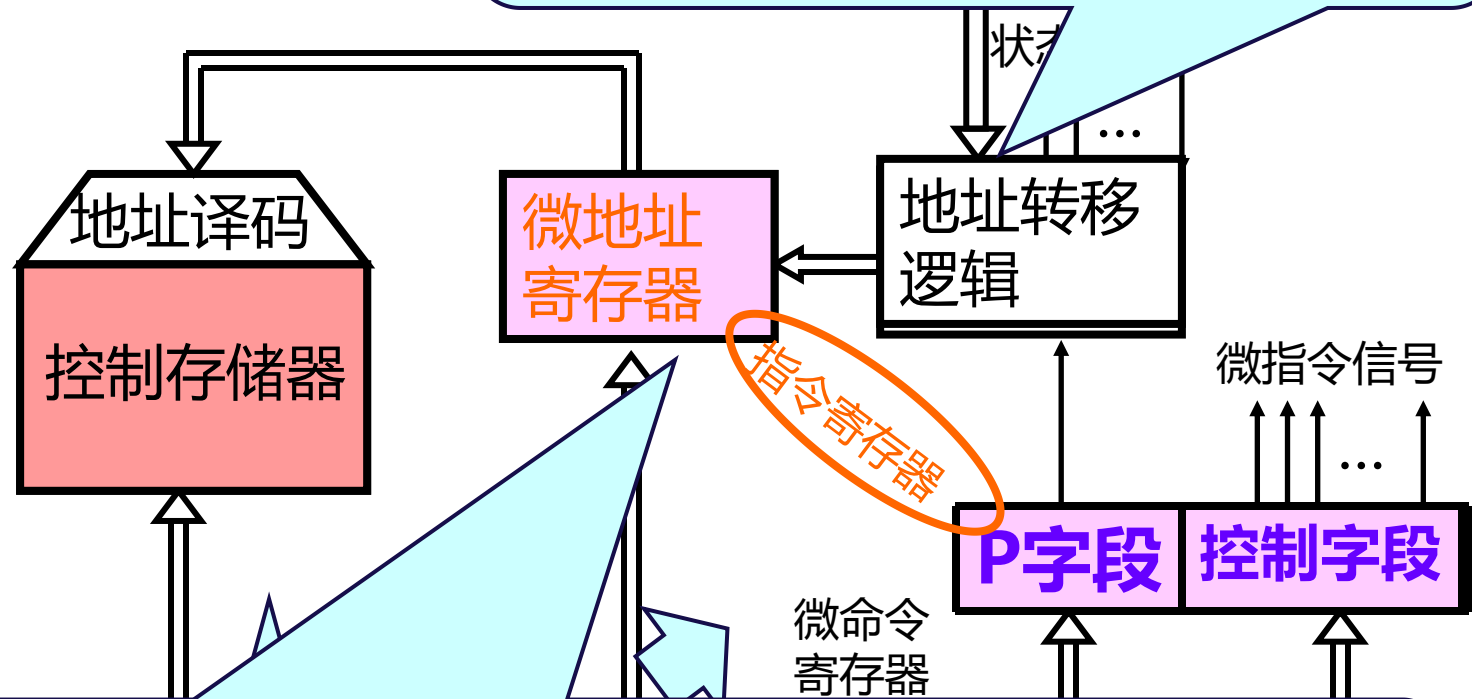
微程序控制器原理框图



微程序控制器组成原理框图

地址转移逻辑承担自动完成修改微地址的任务

如果微程序不出现分支，那么下一条微指令的地址就直接有微地址寄存器给出。当微程序出现分支时，意味着微程序出现条件转移。



微指令寄存器：它用来存放由控制存储器读出的一条微指令信息。
其中**微地址寄存器**：决定将要访问的下一条微指令的地址，
微指令寄存器：保存一条微指令的操作控制字段和判别测试字段的信息。

4、微命令的编码方法

微指令编码：对微指令中操作控制字段采用的表示方法。

编码有三种方法：直接表示法/编码表示法/混合表示法

- **直接表示法：**操作控制字段中的各位分别可以直接控制计算机，不需要进行译码。
- **编码表示法：**将操作控制字段分为若干个小段，每段内采用最短编码法，段与段之间采用直接控制法。
- **混合表示法：**将前两种结合在一起，兼顾两者特点。一个字段的某些编码不能独立地定义某些微命令，而需要与其他字段的编码来联合定义。

5、微指令格式

分为两类：水平型微指令和垂直型微指令

◆ 水平型微指令

- 水平型微指令是指一次能定义并能并行执行多个微命令的微指令。格式如下



水平型微指令又分为三种：

第一种是全水平型（不译码）微指令

第二种是字段译码法水平型微指令；

第三种是直接和译码混合的水平型微指令

◆ 垂直型微指令：采用编码方式。

- 设置微操作控制字段时，一次只能执行一到二个微命令的微指令称为垂直型微指令。



6、程序设计技术有两种

静态微程序设计：

对应与一台计算机的机器指令只有一组微程序，而且这一组微程序设计好之后，一般无需改变而且也不好改变，这种微程序设计技术称为静态微程序设计。

动态微程序设计：

当采用EPROM作为控制存储器时，还可以通过改变微指令和微程序来改变机器的指令系统，这种微程序设计技术称为动态微程序设计。

第八章 输入输出系统

1、I/O接口与端口

- I/O接口是由半导体介质构成的逻辑电路
 - 端口：端口是接口电路中能被CPU直接访问的寄存器。
 - 在一个接口电路中一般拥有：命令端口、状态端口、数据端口
 - 1、数据端口：对数据起缓冲作用
 - 2、状态端口：存放外设或端口本身状态
 - 3、控制端口：存放CPU发出的命令（命令端口）

2、两种I/O端口的编址方式

存储器映像的I/O寻址（统一编址）

存储单元和I/O端口的地址统一编址

I/O映像的I/O寻址（独立编址或单独编址）

I/O端口地址与存储单元地址分开编址

3、CPU与I/O接口之间的数据传送

- 无条件传送方式（简单I/O方式）
- 程序查询（轮询）方式
- 程序中断方式
- 直接内存访问（DMA）方式
- 通道和I/O处理器

4、中断的基本概念

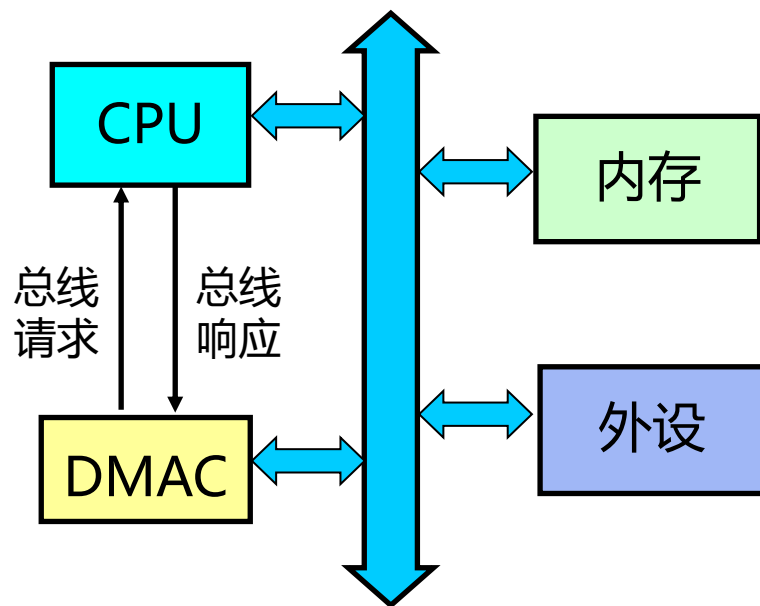
中断 (Interrupt) 某一外设的数据准备就绪后，它“主动”向CPU发出请求中断的信号，请求CPU暂时中断目前正在执行的程序而进行数据交换。当CPU响应这个中断时，便暂停运行主程序，并自动转移到该设备的中断服务程序。

中断源：能够向CPU发出中断请求的事件。

中断向量：中断例行程序的入口地址，存放于中断向量区。

5、DMA的基本概念

- 通过硬件控制实现主存与I / O设备间的直接数据传送，在传送过程中无需CPU的干预。数据传送是在DMA控制器控制下进行的。



- (1)从外围设备**发出DMA请求**;
- (2)CPU响应请求，把CPU工作改成DMA操作方式，**DMA控制器从CPU接管总线的控制**;
- (3)由**DMA** 控制器对内存寻址，即决定数据传送的内存单元地址及数据传送个数的计数，并执行数据传送的操作;
- (4)向 CPU 报告DMA 操作的结束。

请各位同学认真复习，有问题可以及时提出。



积极复习备考才是以不变应万变的最佳方法！！

复习课到此结束

Any Questions ?

